MEMORANDUM

To:	Professor Brendon Anderson, Mechanical Engineering, California Polytechnic State University, bga@calpoly.edu
From:	Gabriel Choi, Carmen Alaichamy
Date:	April 29, 2026
Subject:	Lab 3A: Step Response Test

Introduction

The objective of this lab was to characterize the dynamic response of a DC motor using a step input. By applying a constant voltage to the motor and measuring its velocity response over time, the steady-state gain and time constant of the system were determined. These parameters were then used to develop a first-order model of the motor and validate it through comparison with experimental data.

Methodology

The experiment was conducted using an STM32 microcontroller connected to a DC motor and rotary encoder. The provided MicroPython modules (motor.py and encoder.py) were uploaded to the microcontroller using Thonny.

A main program (main.py) attached in Appendix A was developed to perform a step response test. The motor was driven at a constant input voltage percentage (100%), corresponding to a maximum voltage of 24 V from the motor driver. The encoder was used to measure the angular velocity of the motor shaft.

Velocity data was sampled at a fixed rate using the microcontroller's internal clock. The utime.ticks_us() function was used to track elapsed time and ensure consistent sampling intervals. The measured velocity data was stored in queues and later transmitted to a PC via serial communication.

The collected data was plotted using Python in a Jupyter Notebook to obtain the motor's step response. From this response, the steady-state gain and time constant were estimated. A first-order transfer function model was then created and implemented in Simulink to compare the simulated response with experimental results.

Results

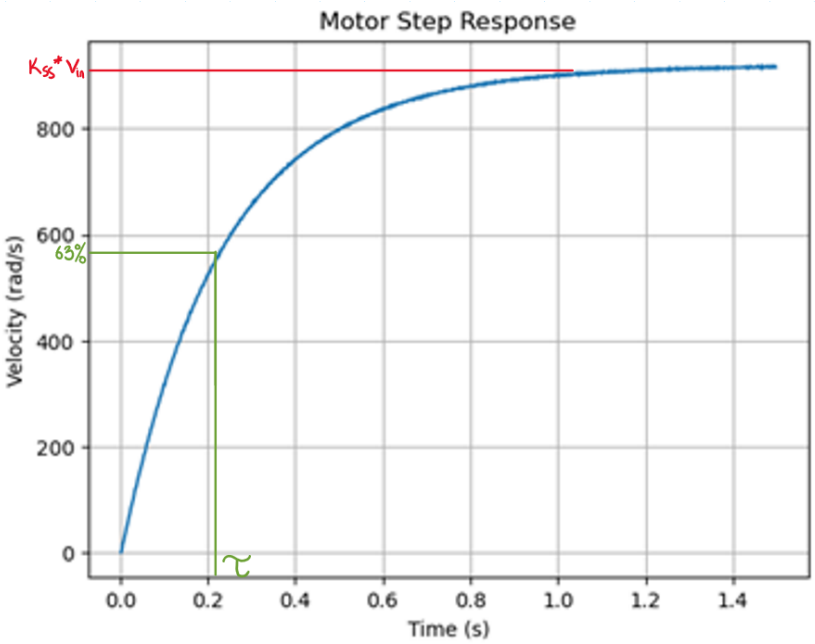


Figure 3A.1. Experimental Velocity [rad/s] Vs. Time (s) Motor Response

Figure 1 shows the experimental step response of the DC motor. The velocity increases rapidly at first and then gradually approaches a steady-state value, exhibiting the characteristic behavior of a first-order system.

From the plot, the steady-state velocity was approximately 925 rad/s. Using an input voltage of 24 V, the steady-state gain was calculated as:

$$K_{ss} = \frac{925}{24} = 38.5 \frac{\frac{rad}{s}}{V}$$

The time constant τ was determined by identifying the time at which the response reached approximately 63% of the steady-state value. This occurred at approximately 0.23 seconds.

The settling time, defined as the time required for the response to remain within 2% of the steady-state value, was estimated to be approximately 1.0 seconds.

Table 3.A.1. Motor Parameters

Steady State Gain K_{ss}	Settling Time t_s (s)	Time Constant τ (s)
38.5	≈ 1.0	0.23

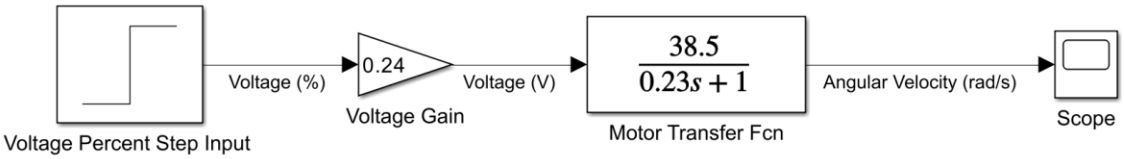


Figure 3A.2. Simulink model of DC motor step response

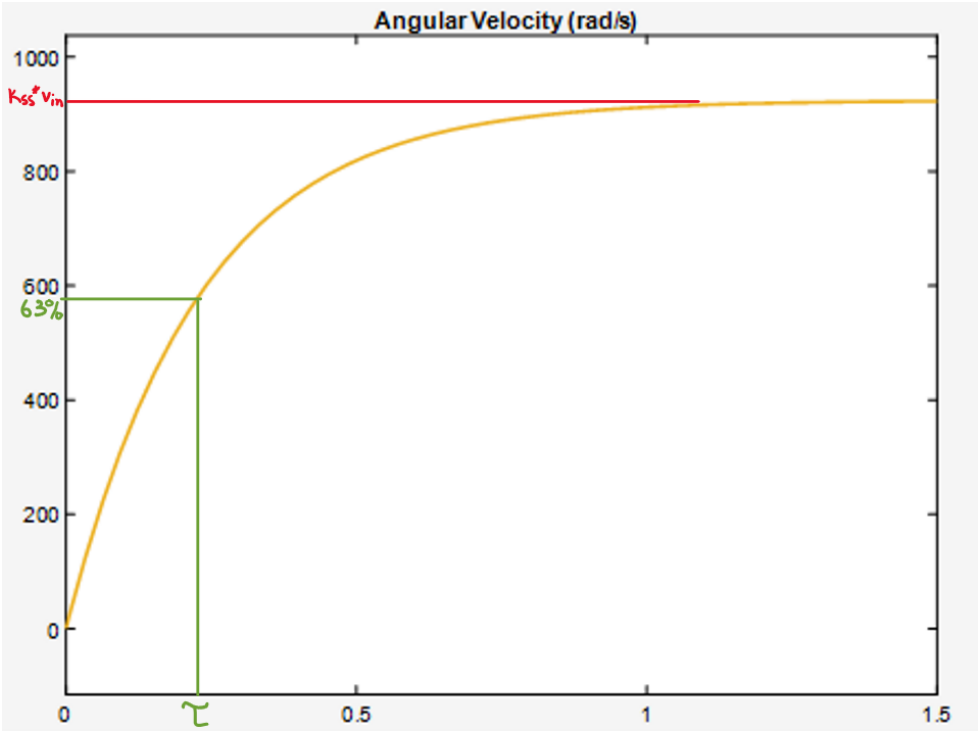


Figure 1. Simulated Velocity [rad/s] Vs. Time (s) Motor Response

Discussion

The magnitude of the step input applied to the motor was 24 V, corresponding to a 100% voltage input to the motor driver.

The sampling rate of the experiment was 500 Hz, meaning velocity data was collected every 2 milliseconds.

Accurate timing during data collection was ensured using the microcontroller's internal clock through the utime module. The function utime.ticks_us() was used to track the current time in microseconds, and utime.ticks_diff() was used to compute the elapsed time between samples. Data was recorded only when the elapsed time exceeded the predefined sampling interval, ensuring a consistent sampling rate throughout the experiment.

```python
import cqueue   # data storage in a queue (from Dr. Ridgely)
import utime    # time related functions
import pyb      # functions related to the Nucleo board
import encoder  # quadrature encoder module
import motor    # motor, motor driver module

# ------------------------------------------------------------
# Initialization
# In this segment, you shall initialze the motor, encoder,
# and constants in the program. Then you shall create queues
# to store the time and velocity test data.

# ------------------------------------------------------------

# WRITE YOUR CODE HERE to create a motor object and enable the motor
# driver on motor A (see motor.py comments)
# Create a MotorDriver object
driver = motor.MotorDriver()
    # Enable motor A
driver.motorA.enable()

print('MOTOR INITIALIZED.')

# 2. WRITE YOUR CODE HERE to create the encoder object and zero the encoder.
# (see encoder.py comments)
  # Create encoder object
enc = encoder.Encoder(4, pyb.Pin.cpu.B6, pyb.Pin.cpu.B7)
    # Zero out encoder
enc.zero()

print('ENCODER INITIALIZED.')

# Constants relating to the timing of the test.
SAMPLING_PERIOD_US = 2000 # The sampling period, in microseconds.
TEST_TIME_US =  1500_000 # The total time of the test, in microseconds.
DATA_POINTS = TEST_TIME_US // SAMPLING_PERIOD_US

# WRITE YOUR CODE HERE to create queues to store data. The time queue is done
# for you. The velocity queue needs to be a FloatQueue.
time_queue = cqueue.IntQueue(DATA_POINTS)
velocity_queue = cqueue.FloatQueue(DATA_POINTS)


# ------------------------------------------------------------
# Main loop
# In this segment, you shall complete the code for step response
# test. In the main loop, the program will sample motor velocity
# periodically, storing it as well as the associated time into the
# data queues declared previously.
# ------------------------------------------------------------

# 1. WRITE YOUR CODE HERE to turn the motor on.
driver.motorA.set_voltage_percent(100)

# These variables are used to run the timed loop. The loop should run at the
# sampling period above and for the total test time above.
start_time = utime.ticks_us()
next_sampling_time = utime.ticks_add(start_time, SAMPLING_PERIOD_US)
current_time = start_time

# 2. Main Loop (timed)
# Runs while current time - start time <= total time
while utime.ticks_diff(utime.ticks_us(), start_time) <= TEST_TIME_US:
    # WRITE YOUR CODE HERE to read the time, using utime.ticks_us().
```

```
    current_time = utime.ticks_us()

    # Use a "if" statement to maintain the constant sampling rate. You will
    # need to compare the time difference between current_time and
    # next_sampling_time using the "utime.ticks_diff" function.
    # If the difference is positive, then enough time has passed and data
    # should be collected.
    # WRITE YOUR CODE HERE to maintain sampling rate with an if-statement.
    if utime.ticks_diff(current_time, next_sampling_time) >= 0:

        # INSIDE OF THE IF STATEMENT
        # WRITE YOUR CODE HERE to update the next sampling time.
        next_sampling_time = utime.ticks_add(next_sampling_time, SAMPLING_PERIOD_US)

        # WRITE YOUR CODE HERE to read the motor velocity, using the encoder.
        velocity = enc.get_velocity_rad()

        # WRITE YOUR CODE HERE to put the data in the queues.
        # Example: time_queue.put(current_time)
        velocity_queue.put(velocity)
        time_queue.put(current_time)
    # Placeholder for empty while loop. Delete after adding your code.


# ------------------------------------------------------------
# Finishing
# In this part, you shall turn the motor off and transfer
# data to the computer.
# ------------------------------------------------------------

# 3. WRITE YOUR CODE HERE to turn the motor off.
driver.motorA.set_voltage_percent(0)
# 4. UNCOMMENT THE CODE BELOW to print the data.
# Be careful with indentation. You may need to replace the variables
# "time_queue" and "velocity_queue" with whatever you called those.

while time_queue.any():
    print(utime.ticks_diff(time_queue.get(), start_time) / 1e6,
    velocity_queue.get(), sep=',')
    utime.sleep_ms(1) # constant printing can cause errors in Thonny
print('END')
```